

Table for Four

A dining concierge you talk to. It understands what kind of evening you want, finds real restaurants, checks what they're known for, applies an offer if one fits, and books the table — remembering you for next time.

THE PRODUCT

A booking journey, not a search box

The concierge is called **Dino**. A guest describes an evening in their own words — "*birthday dinner Friday, four of us, one guest is gluten-free*" — and Dino runs the whole errand: works out what's missing, asks for it, shortlists restaurants, flags which carry an offer, checks real availability, reads the details back for a yes, and books.

The interesting part is what it *won't* do. It never invents a restaurant, a time, an email address or a confirmation number. Every fact it states came back from a tool, and the awkward rules are enforced in code rather than requested politely in a prompt.

GUARDRAIL

Dining only. Anything else gets one warm sentence and a redirect — it will not answer the off-topic question even partially.

GUARDRAIL

An email must appear in the guest's own typed messages before it can be saved. The model cannot supply one from imagination.

GUARDRAIL

Only restaurants actually shown to the guest can be booked, at exactly the time they picked — not a nearby free slot.

GUARDRAIL

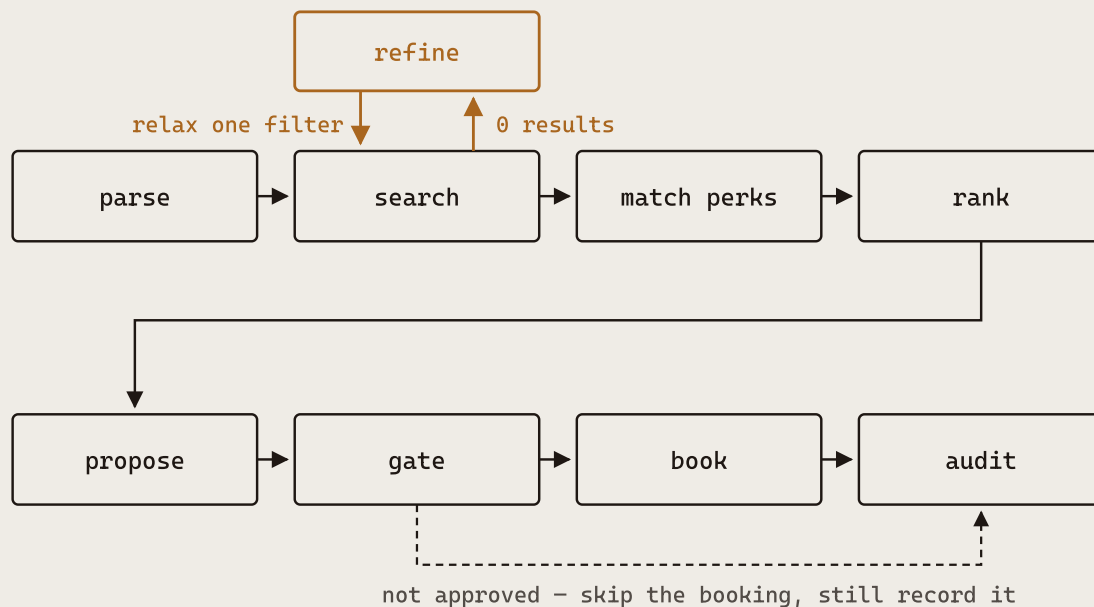
Cancellations inside 24 hours are refused by the backend, which returns the restaurant's phone number instead. Dino relays it and never claims otherwise.

The rule of thumb throughout: if getting it wrong would embarrass the guest at the table, it belongs in code, not in the prompt.

ARCHITECTURE

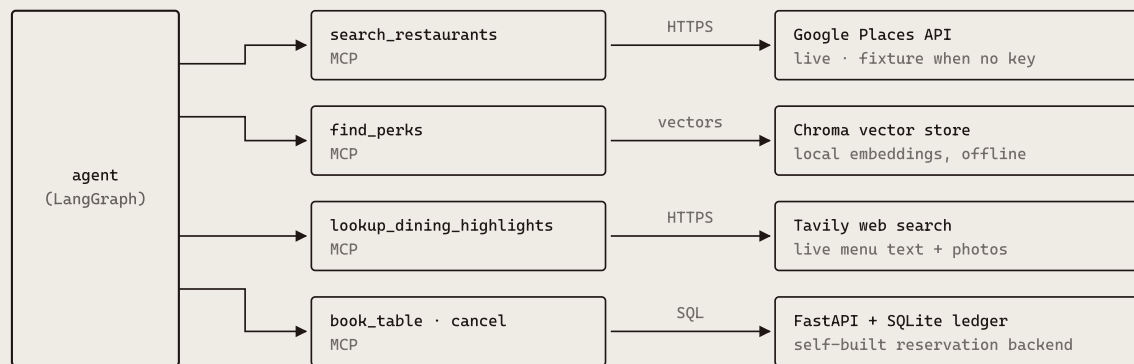
LangGraph decides; MCP servers act

The reasoning loop is a **LangGraph state machine** — an explicit graph of nine steps rather than a model improvising tool calls. Every node reads and writes one shared state object, so at any moment you can see exactly what the agent knows and why it moved.



The refine edge (amber) is what makes this a reasoning loop rather than a pipeline: an empty search sends the agent back to relax one constraint and try again, bounded at two attempts so it can never spin.

Everything the agent can actually *do* lives behind **MCP servers** — four small, separately testable tool servers. The graph never touches an API or a database directly; it asks a tool. That boundary is what makes the whole thing testable offline and auditable in production.



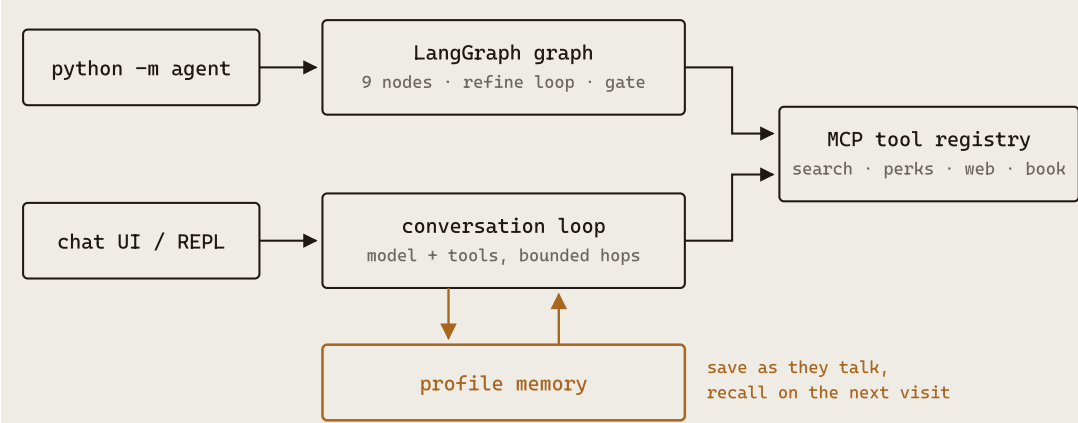
Two sources are live (Places, Tavily) and two are local (the vector store, the reservation ledger). Every tool result is tagged with which — so an audit can tell whether a booking rested on real data or a fixture.

Booking is deliberately a **self-built mock backend**: OpenTable, Resy and SevenRooms are all partner-gated. It's a real FastAPI service over a real SQLite ledger — restaurant, time, party, guest, status, cancellation stamps — so the transactional half behaves like a system of record rather than a stub.

Two orchestrators, one set of tools

There's a second loop worth naming, because it's the one the demo actually runs on. The graph above is the **end-to-end orchestrator**: hand it a request and it goes from sentence to booking on its own. The **conversation loop** is different — it's turn-by-turn, the guest stays in the loop at every choice, and each turn is a bounded round of model-then-tools.

They are parallel: neither calls the other, and both reach the world through the same MCP registry. **Long-term memory hangs off the conversation loop only** — which is why there's no "look up profile" node in the graph. The graph is stateless about who you are; the concierge is the part that remembers.



The same tools, driven two ways. Only the conversation loop touches the guest's profile (amber) — the graph knows about the request in front of it and nothing about who is asking.

MEMORY & RETRIEVAL

Three kinds of memory, three kinds of retrieval

"Memory" gets used loosely. Here it's three separate things with three different lifetimes, and mixing them up is what makes agents feel broken.

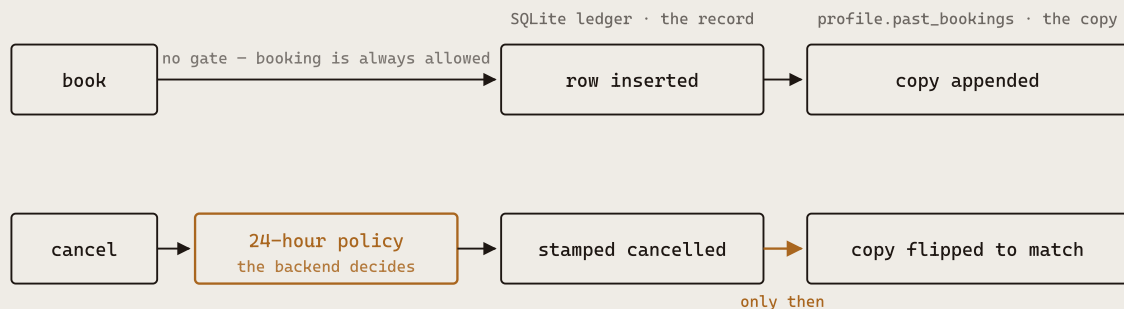
TIER	HOLDS	LIVES IN	LASTS
Working	The outing being planned — party size, date, chosen restaurant	Session state, restated on every tool result	The conversation
Long-term	Who the guest is — name, pronouns, email, cuisines, dietary needs, past bookings	Chroma, keyed by email	Across visits
Ledger	What was actually booked and cancelled	SQLite, in the backend	Permanent record

The working tier earns its keep in a subtle way: every tool result carries back a `known_so_far` block. Long chats drift, and a model that has to remember an email from twenty turns ago will eventually ask for it twice. Handing the facts back on every call removes the chance to forget.

Long-term memory has a deliberate asymmetry. Cuisines are a **rolling window of the last three** — taste drifts, and a guest who has booked Italian three times running shouldn't still be filed as a sushi person from one visit last year. Dietary needs are never aged out: an allergy is not a preference. And remembering someone is treated as licence to *offer*, not to decide — Dino asks whether you want your usual, something near you, or something new.

Where the profile meets the ledger

Two of those tiers describe the same reservation, which raises the obvious question: which one is right? The **ledger is the system of record** — it decides what exists. The profile keeps a small copy so Dino can greet you with "your table at Osteria last month" without querying the backend mid-sentence. They're joined by the **confirmation id**.

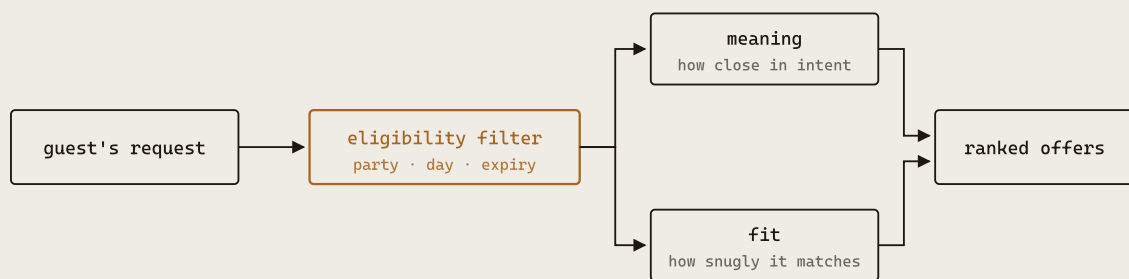


The ordering is the point. A cancellation is decided by the backend — inside 24 hours it refuses and hands back the restaurant's phone number — and the profile copy is only updated once the ledger has actually changed. Memory follows the record; it never leads it.

That sync needs its own write path. The union-merge that's right for preferences would *append* a second copy of the booking rather than update the existing one, so flipping a status is a targeted in-place edit instead. It's a small thing that would quietly corrupt a guest's history if it were wrong — a cancelled table still showing as booked, next visit.

Retrieval, where it actually helps

Perks — the offers attached to restaurants — are the RAG surface. They're private data the model has never seen, written as free text, and matched by meaning: "*celiac-safe birthday*" has to find "*gluten-free tasting flight for celebrations*" with barely a word in common. That's retrieval's job, not a prompt's.



Eligibility (amber) is never left to the embedding. An expired offer, or one needing a bigger party, is dropped *before* ranking — so a perfect semantic match can't become a discount the guest is unable to claim. What survives is ordered by $0.7 \times \text{meaning} + 0.3 \times \text{fit}$: meaning-dominant, because fit decides whether an offer *can* be used while meaning decides whether it's the right one to mention.

1.00 1.00 24 104

HIT @ 3 MRR PERKS INDEXED TESTS PASSING

Measured against a labelled benchmark of ten realistic requests, each paired with the restaurant a good retriever should surface. The blend weight was chosen by sweeping it and keeping what scored best — and guard tests fail the build if a change makes retrieval worse.

And a third retrieval surface: the live web

Perks and profiles are ours; what a restaurant is actually *known for* isn't. A Tavily search fetches cited menu highlights and photos on demand — but pointed only at a restaurant already on the table, so it can't become a general web-

search back door around the dining-only rule. It searches the restaurant's own site first, and prefers photos lifted from the pages that matched, which is what makes a chain return the right branch rather than a stock plate of food.

THE SURFACE

What the guest sees

The chat UI is **Streamlit**, deliberately thin — it wraps the same session API the terminal version uses, so the conversation logic has exactly one home. Two things make it feel like a product rather than a transcript.

MEMORY, MADE VISIBLE

A side panel shows the guest's long-term profile filling in live as they talk — name, cuisines, dietary needs, past bookings — so the thing the system claims to remember is on screen, not asserted.

MENU CARDS

Each restaurant gets a generated, cuisine-themed card carrying the dishes retrieved from the web and the perk, with a note beside it in italic. Six themes, drawn as SVG — no image files, works offline.

The cards have one rule that matters more than how they look: **every dish on them is a substring of text actually retrieved**. When a restaurant's menu isn't published online, the card says so rather than inventing something plausible. Reservations appear as cards with a downloadable calendar invite, and a perk triggers a small celebration.

The stack, in one list

Orchestration	LangGraph state machine with a checkpointer per conversation thread
Tools	Four MCP servers — search, perks, web highlights, booking
Model	Provider-agnostic: OpenAI or OpenRouter, one config switch
Retrieval	Chroma + local MiniLM embeddings; Tavily for live web
Data	Google Places (live), FastAPI + SQLite reservation ledger (self-built)
Interface	Streamlit chat, terminal REPL, LangGraph Studio
Quality	104 offline tests; a labelled retrieval benchmark; LangSmith tracing

The whole system runs **offline with no API keys at all** — fixtures for search, a local embedding model, an in-process backend, placeholder graphics — and switches to live data the moment a key appears. The same code path runs in both, which is why the demo can't quietly diverge from the real thing.

Still to come: the explicit human-approval gate before any booking is confirmed, and the governance trail recording what the agent did, on whose data, and why.

Built with Claude Code — an agent, used to build an agent.